

DSA STUDY BLUEPRINT SERIES

61. Introduction to STACKS

Last-In-First-Out Data Storage Framework

Section 08 • C++ Core Data Structures & Algorithms

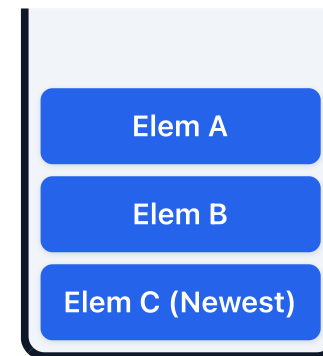
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Stack:** LIFO storage structure. Operations: push, pop, top.
- Can be implemented using dynamic arrays or linked lists.

Memory & Execution Blueprint



LIFO Stack beaker (Top points to Elem C)

C++ Implementation Reference

Standard code layout and syntax

```
class Stack {  
    int arr[1000], topIdx = -1;  
public:  
    void push(int x) { arr[++topIdx] = x; }  
    void pop() { topIdx--; }  
    int top() { return arr[topIdx]; }  
    bool empty() { return topIdx == -1; }  
};
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Push / Pop / Top	$O(1)$
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** Always check for stack overflow and underflow before modifying indices.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace monotonic stack modifications for next greater element on `[4, 5, 2]` :

Active Element	Stack State (Indices)	Pop comparison	Result output
4	[0] (value 4)	N/A	Waiting for next greater
5	[1] (value 5)	5 > 4 (pop index 0)	Next Greater of 4 is 5

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Monotonic properties:** Stacks process elements strictly increasing or decreasing, optimizing nested lookups.
- **Min Stack details:** Track current minimum bounds by storing elements as encoded difference values.
- **Related LeetCode Problems:** #155 Min Stack, #84 Largest Rectangle in Histogram, #42 Trapping Rain Water.